المدرسة الوطنية للعلوم التطبيقية - بني ملال
Ecole Nationale des Sciences Appliquées - Béni Mellal

Rapport de Travaux Pratiques

TP3 — Architecture Distribuée

Réalisée par	Nawar El Asraoui
Filière	Cybersécurité et Intelligence Artificielle
Encadré par	Pr. BE ELBAGHAZAOUI
Année universitaire	2026 — 2027

Mars 2026

1. Présentation du système

Ce TP a pour objectif de déployer une **architecture distribuée complète** composée de six composants interconnectés, tous orchestrés via Docker Compose. Le flux de données suit le chemin : **Client → Nginx → Node.js → MongoDB / Redis / RabbitMQ**.

Composant	Technologie	Rôle
Client	Postman	Envoie les requêtes HTTP POST/GET
Load Balancer	Nginx :80	Répartit le trafic entre les deux instances Node.js
Serveur	Node.js / Express	Implémente la logique métier
Base de données	MongoDB :27017	Stockage persistant des commandes
Cache	Redis :6379	Mise en cache des résultats GET (TTL 30s)
Bus de messages	RabbitMQ :5672	Notifications asynchrones après chaque commande

2. Déploiement et état des conteneurs

2.1 Conteneurs Docker

La figure ci-dessous montre l'état de tous les conteneurs après le lancement du système via docker-compose up --build. On observe que nginx, redis, rabbitmq et mongodb sont actifs. Les instances app1 et app2 ont été redémarrées après l'initialisation complète de RabbitMQ afin d'éviter les erreurs de connexion au démarrage.

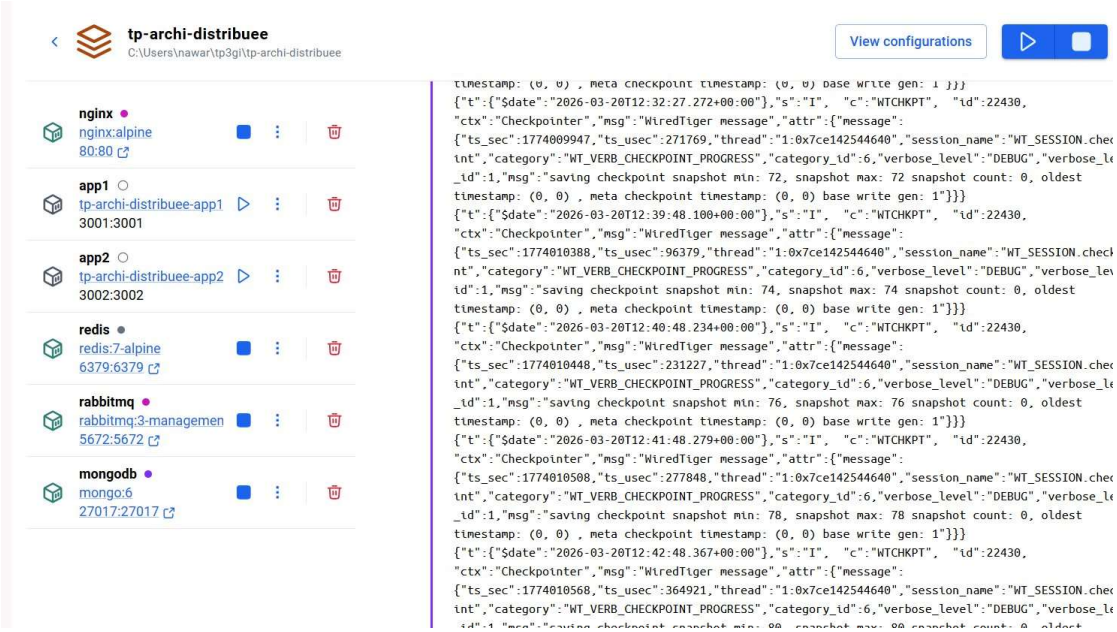

Figure 1 — Docker Desktop : état de tous les conteneurs

2.2 Logs de l'instance app1

La figure ci-dessous confirme que l'instance app1 a établi avec succès toutes ses connexions aux services (MongoDB, Redis, RabbitMQ) et qu'un message a été publié dans la queue RabbitMQ suite à la création d'une commande.

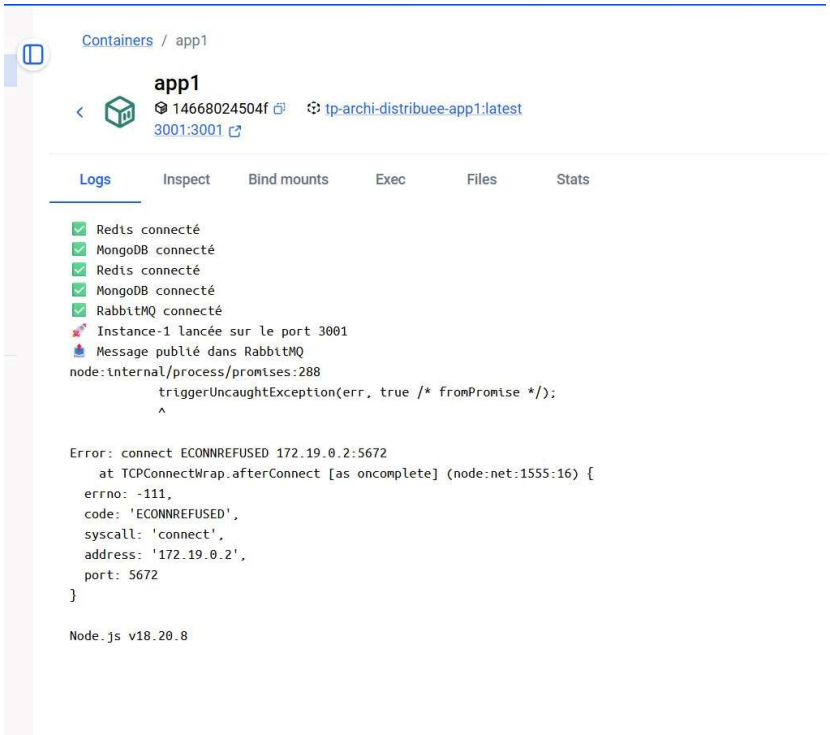

Figure 2 — Logs Docker de app1 : connexions établies et message publié

2.3 Interface RabbitMQ Management

La figure ci-dessous montre le tableau de bord RabbitMQ accessible sur http://localhost:15672. On y observe 2 connexions actives (app1 et app2) et 1 queue nommée commandes correctement créée.

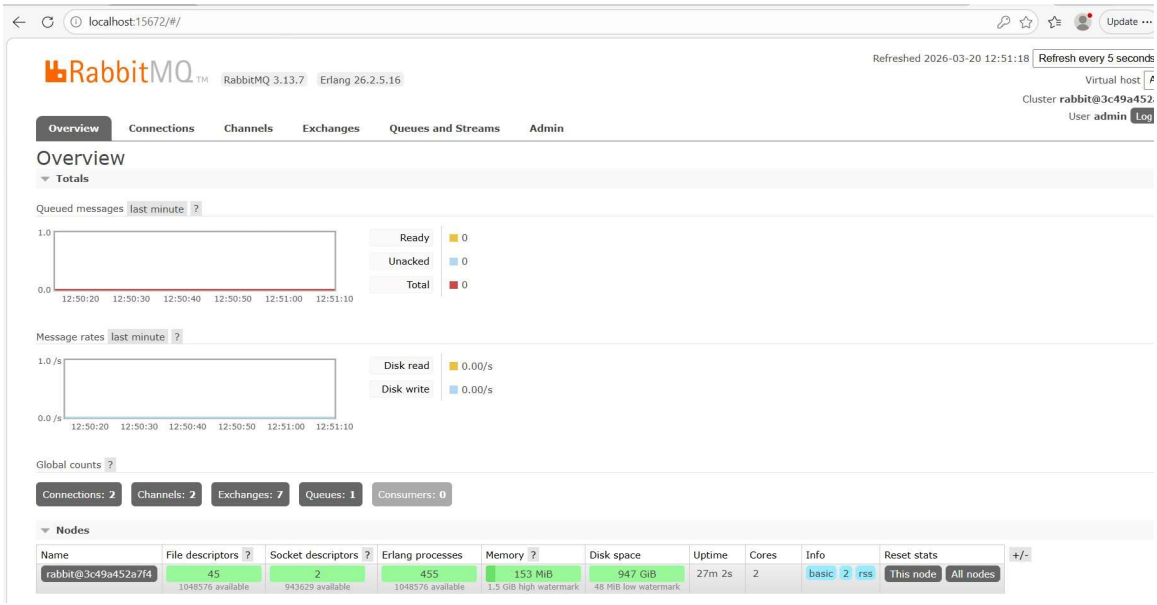

Figure 3 — Interface RabbitMQ Management : 2 connexions, 1 queue active

3. Tests avec Postman

3.1 Test 1 — Création d'une commande (POST)

Une requête **POST http://localhost/commandes** est envoyée avec un corps JSON contenant le produit, la quantité et le client. La réponse retourne le statut **201 Created** et indique que la commande a été traitée par **Instance-1**.

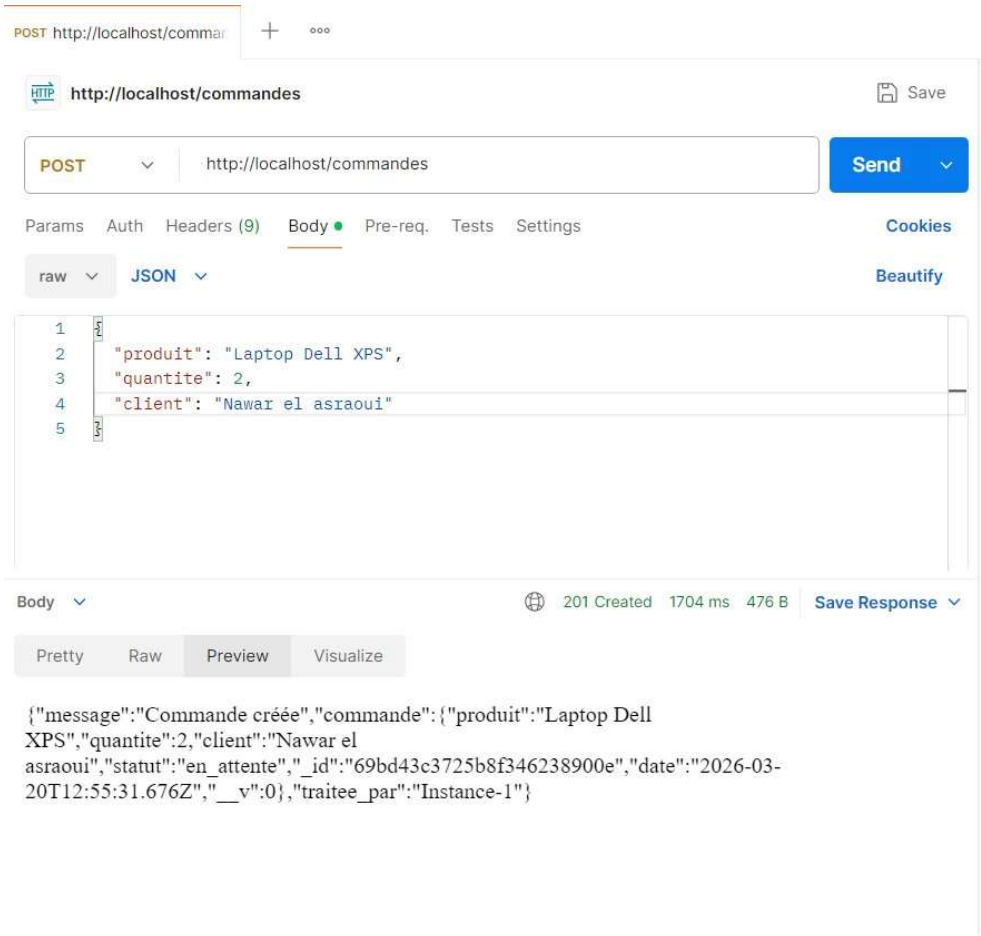

Figure 4 — Test 1 : POST /commandes — Réponse 201 Created, traitée par Instance-1

3.2 Test 2 — Observation du cache Redis (GET)

Premier appel (904 ms) : les données sont lues directement depuis MongoDB. Node.js stocke ensuite le résultat dans Redis avec un TTL de 30 secondes.

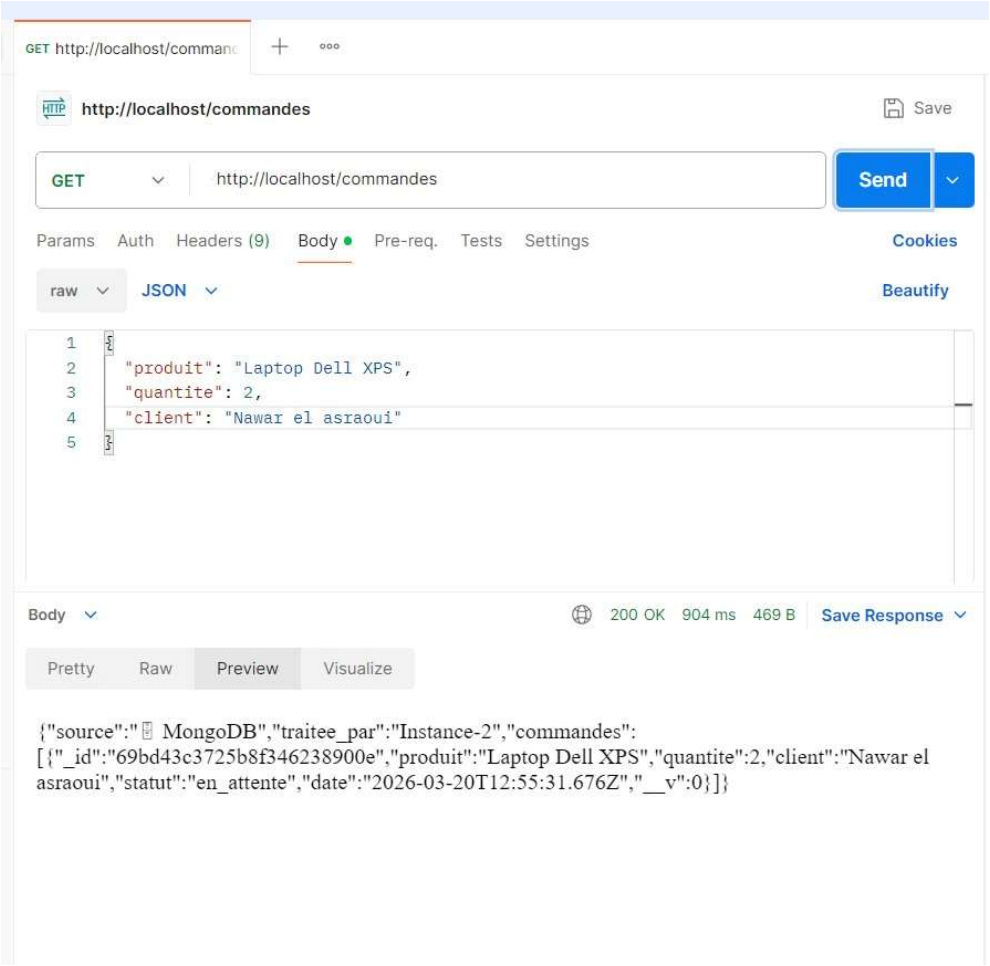

Figure 5 — Test 2 : 1er GET — Source MongoDB (904 ms)

Deuxième appel (26 ms) : les données sont retournées depuis le cache Redis sans solliciter la base de données, soit **35 fois plus rapide**.

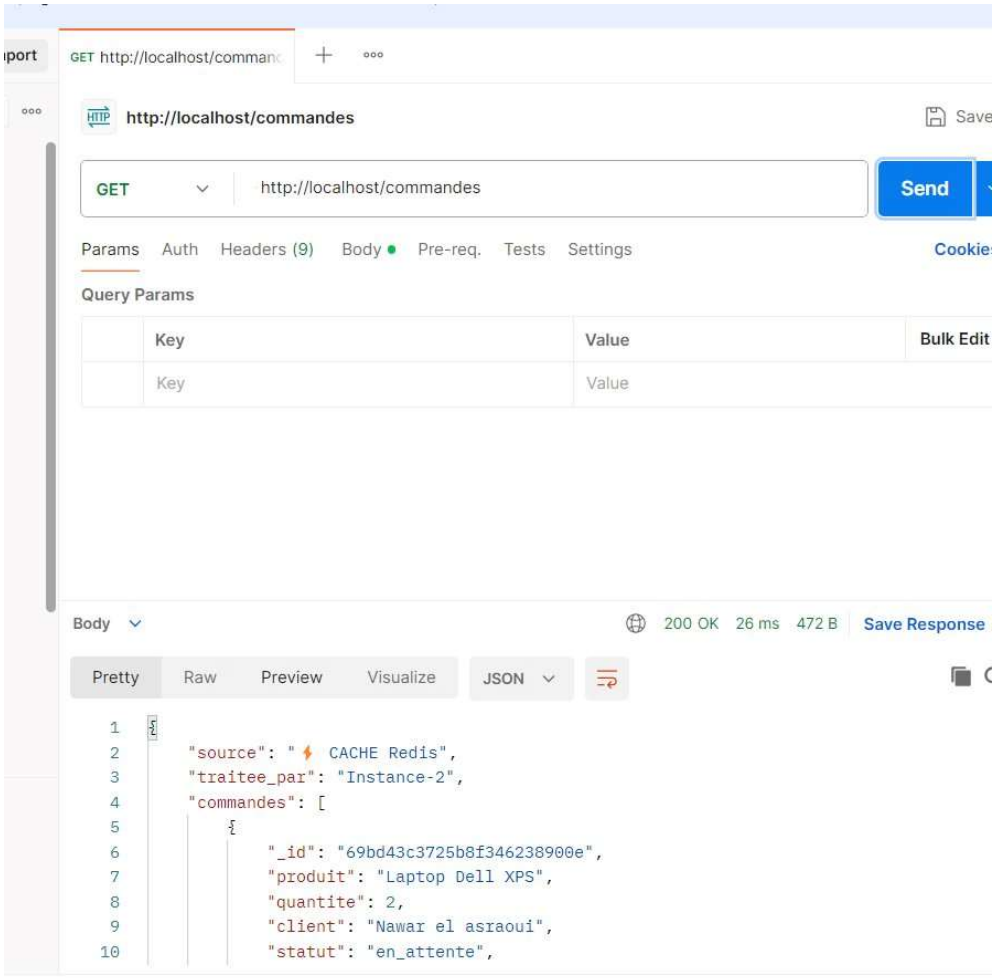

Figure 6 — Test 2 : 2ème GET — Source Cache Redis (26 ms)

3.3 Test 3 — Load Balancing Round-Robin

Quatre requêtes POST successives sont envoyées. Le champ traitee_par alterne entre Instance-1 et Instance-2, confirmant la répartition Round-Robin assurée par Nginx.

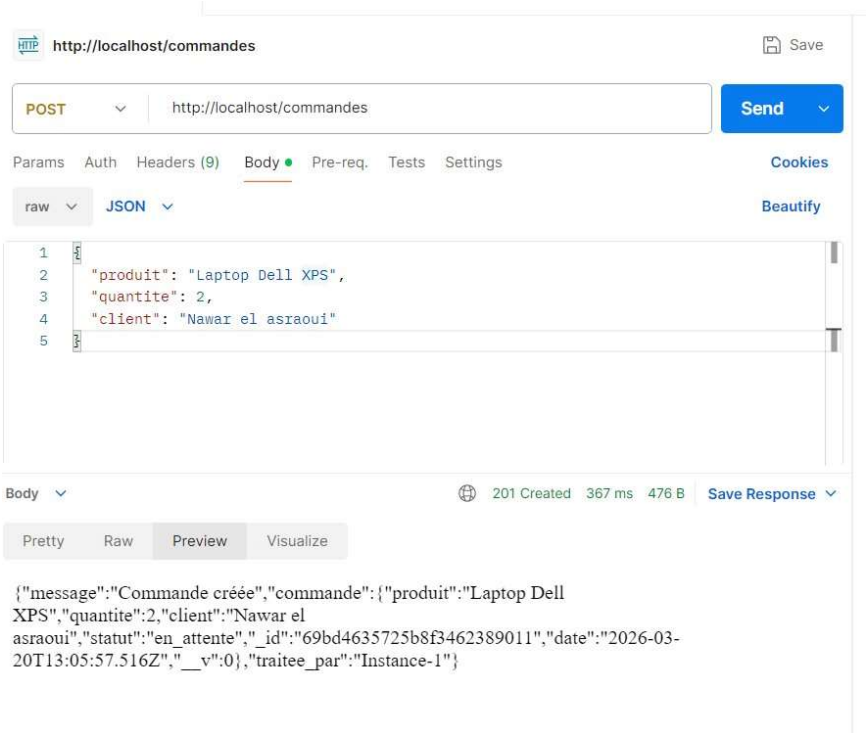

Figure 7 — Requête impaire : traitée par Instance-1

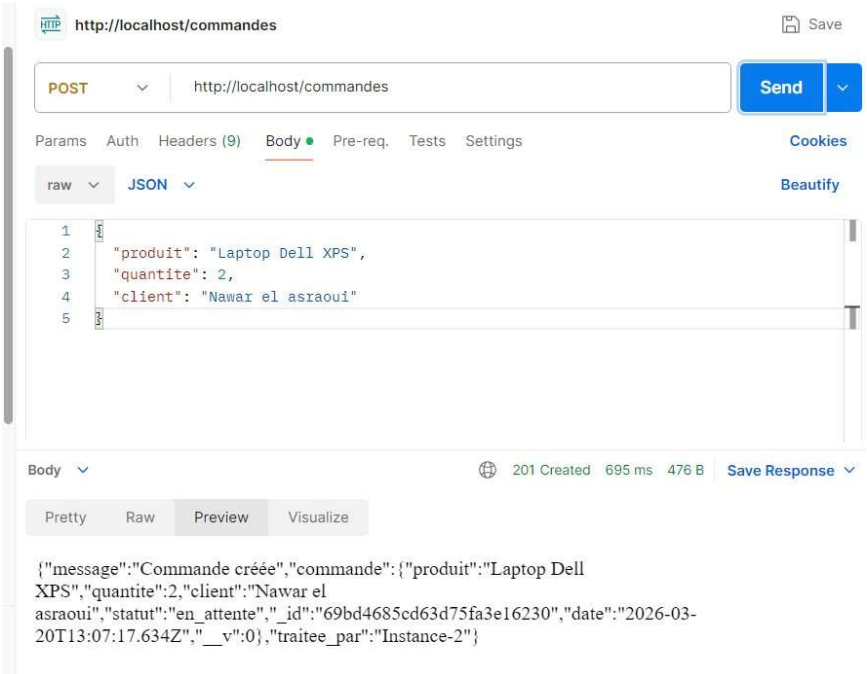

Figure 8 — Requête paire : traitée par Instance-2

Requête	Instance	Status	Observation
1	Instance-1	201 Created	Premier serveur
2	Instance-2	201 Created	Deuxième serveur

3	Instance-1	201 Created	Retour au premier
4	Instance-2	201 Created	Alternance confirmée

4. Réponses aux questions

Q1 — Quel composant reçoit en premier les requêtes ? Quel algorithme utilise-t-il ?

C'est Nginx (port 80) qui reçoit en premier toutes les requêtes du client. Il utilise l'algorithme **Round-Robin** par défaut, qui distribue les requêtes en alternance cyclique entre app1:3001 et app2:3002. Cela a été confirmé par les tests Postman où le champ traitee_par alterne entre Instance-1 et Instance-2 à chaque requête successive.

Q2 — Que se passe-t-il si app1 tombe en panne ? Nginx continue-t-il à fonctionner ? Pourquoi ?

Oui, Nginx continue de fonctionner. **Il dispose d'un mécanisme de détection de pannes passives :** lorsqu'app1 ne répond plus, Nginx le marque comme indisponible et redirige automatiquement toutes les requêtes vers app2:3002, sans interruption de service pour le client. Quand app1 revient en ligne, Nginx reprend la répartition Round-Robin normalement.

Q3 — Expliquez la différence entre le 1er GET (MongoDB) et le 2ème GET (Redis).

Lors du premier appel (904 ms), Node.js ne trouve rien dans Redis, interroge directement MongoDB, puis stocke le résultat dans le cache Redis avec un TTL de 30 secondes. Lors du **deuxième appel (26 ms)**, Node.js trouve les données en mémoire dans Redis et répond instantanément sans accéder à la base de données. Le cache Redis est **35 fois plus rapide** que la lecture directe en MongoDB.

Q4 — Pourquoi invalide-t-on le cache Redis après un POST /commande ?

Après la création d'une nouvelle commande dans MongoDB, le cache Redis contient encore l'ancienne liste sans cette nouvelle entrée. Si on ne l'invalide pas, le prochain GET retournerait des **données obsolètes**. En supprimant la clé toutes_commandes dans Redis après chaque POST, on force le prochain GET à relire les données fraîches depuis MongoDB. C'est la stratégie **Cache-Aside** qui garantit la cohérence des données entre la base et le cache.

Q5 — En quoi RabbitMQ améliore-t-il la résilience du système ?

RabbitMQ introduit un **découplage asynchrone** entre le serveur Node.js et les services de traitement aval. Sans RabbitMQ, Node.js appellerait directement le service de notification de façon synchrone : si ce service est lent ou hors ligne, la requête POST serait bloquée. Avec RabbitMQ, Node.js publie le message dans la queue et répond immédiatement au client sans attendre. Les messages sont persistants (durable: true) : si le consumer tombe en panne, les messages restent dans la queue et seront traités à son redémarrage, améliorant ainsi la disponibilité et la tolérance aux pannes du système.

Q6 — Quelle ligne modifier dans nginx.conf pour une 3ème instance ?

Il suffit d'ajouter la ligne **server app3:3003;** dans le bloc upstream backend du fichier nginx/nginx.conf, et d'ajouter le service app3 correspondant dans docker-compose.yml. Nginx intégrera automatiquement cette troisième instance dans la rotation Round-Robin, répartissant ainsi la charge entre trois serveurs sans aucune autre modification nécessaire.

5. Conclusion

Ce TP a permis de déployer et de valider une architecture distribuée complète s'appuyant sur six composants complémentaires. Les tests réalisés avec Postman ont démontré concrètement trois comportements clés : la persistance des données via MongoDB, le gain de performance significatif du cache Redis (35 fois plus rapide), et la répartition de charge Round-Robin assurée par Nginx entre les deux instances Node.js. L'utilisation de RabbitMQ a permis de découpler les traitements asynchrones, renforçant la résilience du système. Docker Compose a simplifié l'orchestration de l'ensemble en un déploiement unique et reproductible, illustrant les principes fondamentaux des microservices modernes.